

Universidad del Valle de Guatemala
Facultad de Ingeniería, Departamento de Ciencias de la Computación
CC3067 Redes

Proyecto 1

Uso de un protocolo existente

Hugo Ernesto Barillas Villagrán
Carnet 23306

6 de septiembre de 2026

Introducción

En este proyecto se construyó un chatbot que puede hablar con un modelo de lenguaje y que además puede usar herramientas externas por medio del protocolo MCP, MCP quiere decir Model Context Protocol, y es una forma en la que un modelo de lenguaje puede pedirle a un programa externo que haga una tarea, por ejemplo agendar una cita o guardar un archivo en el disco.

Para este proyecto se creó un servidor MCP propio que simula el sistema de citas de una clínica, este servidor permite ver las especialidades que se atienden, ver los doctores de cada especialidad, ver los horarios libres de un doctor, agendar una cita nueva, cancelar una cita y cambiar la fecha de una cita ya existente, este mismo servidor se hizo correr de dos formas distintas, una de forma local en la propia computadora, y otra de forma remota en la nube usando Cloudflare Workers.

Además del servidor propio, el chatbot también se conectó a dos servidores oficiales de Anthropic, uno para manejar archivos y otro para manejar un repositorio de git, y con esto se demostró un caso donde el chatbot crea un archivo, lo agrega a un repositorio y hace un commit, todo por medio de una conversación normal.

Todo el protocolo JSON-RPC que se usa para hablar con los servidores se escribió a mano, sin usar ningún SDK de MCP, esto se hizo así porque el enunciado del proyecto lo pedía de esa forma, la idea de hacerlo a mano es poder entender bien cómo funciona el protocolo por dentro, y no solamente usar una librería que ya resuelve todo.

Funcionalidades implementadas

Conexión con el modelo de lenguaje

El chatbot habla directamente con la API de Anthropic, se le puede preguntar cualquier cosa de conocimiento general y responde usando lo que el modelo ya sabe, sin necesidad de ninguna herramienta externa.

Contexto de la conversación

El chatbot recuerda todo lo que se ha hablado en la misma sesión, si se le pregunta algo y luego se hace una segunda pregunta relacionada, el chatbot entiende a qué se refiere esa segunda pregunta, esto se logra guardando el historial completo de la conversación en memoria mientras el programa sigue corriendo.

Registro de las llamadas a los servidores MCP

Cada vez que el chatbot le pide algo a un servidor MCP, esa solicitud y la respuesta que llega se muestran en la pantalla y también se guardan en un archivo de texto, así queda un registro completo de todo lo que se conversó con cada servidor durante la sesión.

Servidores oficiales de Anthropic

El chatbot se conecta al servidor oficial de archivos y al servidor oficial de git, con esto se puede pedir al chatbot que cree un archivo nuevo, que lo agregue a un repositorio, y que haga un commit, todo esto se probó de forma real y funcionó correctamente.

Servidor propio de forma local

Se creó un servidor MCP para la clínica, este servidor corre dentro de la misma computadora donde corre el chatbot, y se comunica con el chatbot escribiendo y leyendo texto por la entrada y salida estándar.

Servidor propio de forma remota

El mismo servidor de la clínica se subió a Cloudflare Workers, que es un servicio de nube, para que el chatbot lo pueda usar desde internet igual que usa el servidor local, la diferencia principal es que en vez de comunicarse por entrada y salida estándar, se comunica por internet usando HTTP.

Análisis con Wireshark

Se capturó el tráfico que viaja entre el chatbot y el servidor remoto, para poder ver los mensajes JSON-RPC que se mandan y clasificarlos según su tipo, este análisis se explica con más detalle más adelante en este documento.

Especificación de los servidores

Servidor local, `mcp_server_v1`

Este servidor corre en la computadora usando Python, no usa ninguna librería externa, solamente lo que ya trae Python, se comunica con el chatbot por medio de la entrada y salida estándar, es decir que el chatbot le manda un mensaje de texto y el servidor contesta con otro mensaje de texto, el formato de esos mensajes es JSON-RPC 2.0.

Este servidor tiene seis herramientas disponibles.

`list_specialties` no necesita ningún parámetro, devuelve la lista de especialidades que atiende la clínica, que son cardiología, dermatología, pediatría y medicina general.

`list_doctors` necesita el parámetro `specialty`, devuelve los doctores que atienden esa especialidad.

`check_availability` necesita `doctor_id`, `date_from` y `date_to`, con formato de fecha año mes día, devuelve los horarios libres de ese doctor en ese rango de fechas.

`book_appointment` necesita `doctor_id`, `patient_name` y `datetime`, con formato año mes día hora minuto, crea una cita nueva y devuelve un número de cita.

`cancel_appointment` necesita `appointment_id`, cancela esa cita y libera el horario.

`reschedule_appointment` necesita `appointment_id` y `new_datetime`, cambia la fecha y hora de esa cita.

Servidor remoto, `mcp_server_remote`

Es el mismo servidor de la clínica, pero escrito en JavaScript en vez de Python, y desplegado en Cloudflare Workers, tiene las mismas seis herramientas con los mismos parámetros que el servidor local, la diferencia está en la forma de comunicarse, el chatbot le manda una petición POST por internet con el mensaje JSON-RPC dentro, y el servidor contesta con otro mensaje JSON-RPC dentro de la respuesta HTTP.

La dirección donde vive este servidor es `https://clinic-appointment-mcp-remote.hugomcp.workers.dev/`

Chatbot

El chatbot es el programa principal del proyecto, está escrito en Python, se conecta a la API de Anthropic para hablar con el modelo de lenguaje, y al mismo tiempo se conecta a varios servidores MCP, al servidor de la clínica local, al servidor de archivos, al servidor de git, y de forma opcional al servidor de la clínica remoto.

Cada herramienta que ofrece un servidor se le muestra al modelo con un nombre que dice de qué servidor viene, de esta forma el chatbot siempre sabe a cuál servidor debe mandarle cada petición que el modelo decide hacer.

Análisis con Wireshark

Para poder ver los mensajes JSON-RPC en Wireshark sin que estuvieran cifrados, se corrió el servidor remoto de forma local usando la herramienta wrangler dev, esto hace que corra el mismo código del servidor remoto pero dentro de la computadora y sin usar HTTPS, se decidió hacerlo así porque al intentar descifrar el tráfico HTTPS real usando la función que trae Python para eso, el programa se cerraba solo por un problema propio de Windows, entonces para poder mostrar el protocolo con claridad se optó por capturar el mismo servidor y el mismo código, solamente sin cifrado.

La captura se hizo usando el filtro `tcp.port==8787`, que es el puerto donde corre el servidor de forma local, y se capturó desde el adaptador de loopback de la computadora, ya que todo el tráfico se queda dentro de la misma máquina.

Mensajes de sincronización

Cuando el chatbot se conecta a un servidor MCP, lo primero que manda es un mensaje llamado initialize, este mensaje le dice al servidor qué versión de protocolo se está usando y quién es el cliente que se está conectando, el servidor contesta con su propia información, incluyendo su nombre y su versión.

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
652	20:19:46.89682	127.0.0.1	127.0.0.1	TCP	72365 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK	
653	20:19:46.89688	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
654	20:19:46.89706	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
655	20:19:46.89718	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD...	
656	20:19:46.89728	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=282 Win=65824 Len=0	
657	20:19:46.89722	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1, 350N (application/json)	
658	20:19:46.89723	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=465 Win=65824 Len=0	
672	20:19:46.10385	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [PSH, ACK] Seq=1 Ack=465 Win=65824 Len=72 [TCP P...	
673	20:19:46.10385	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=73 Win=65280 Len=0	
676	20:19:46.10389	127.0.0.1	127.0.0.1	HTTP/2	HTTP/1.1 200 OK, 350N (application/json)	
677	20:19:46.10390	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=246 Win=65280 Len=0	
678	20:19:46.10407	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [FIN, ACK] Seq=465 Ack=246 Win=65280 Len=0	
679	20:19:46.10409	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=246 Ack=466 Win=65824 Len=0	
680	20:19:46.10411	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [FIN, ACK] Seq=246 Ack=466 Win=65824 Len=0	
683	20:19:46.10413	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=466 Ack=247 Win=65280 Len=0	
684	20:19:46.10579	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK	
685	20:19:46.10580	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
686	20:19:46.10592	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
687	20:19:46.10602	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=280 [TCP PD...	
688	20:19:46.10604	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=281 Win=65824 Len=0	
689	20:19:46.10605	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1, 350N (application/json)	
690	20:19:46.10606	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=352 Win=65824 Len=0	
699	20:19:46.11122	127.0.0.1	127.0.0.1	HTTP	HTTP/1.1 202 Accepted	
700	20:19:46.11124	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=352 Ack=45 Win=65280 Len=0	
703	20:19:46.11149	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [FIN, ACK] Seq=352 Ack=45 Win=65280 Len=0	

Frame 657: Packet, 227 bytes on wire (1816 bits), 227 bytes captured (1816 bits) on interface 0

Null/Loopback

Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1

Transmission Control Protocol, Src Port: 52365, Dst Port: 8787, Seq: 282, Len: 281

[2 Reassembled TCP Segments (464 bytes): #655(283), #657(183)]

Hypertext Transfer Protocol

JavaScript Object Notation: application/json

0000 02 00 00 00 45 00 00 df f1 57 40 00 80 06 0a bfE.WB.....

0001 7f 00 00 01 7f 00 00 01 cc 8d 22 53 ff ff 90 57S.W.....

0002 07 02 04 c2 52 18 00 ff 00 80 00 00 7b 21 6a 73P.....[j

0003 6f 0e 72 70 63 22 3a 20 22 32 30 28 22 2c 20 22onrpc": "2.0", "

0004 69 64 22 3a 20 31 2c 20 22 6d 65 74 68 6f 64 22id": 1, "method"

0005 3a 20 20 69 64 69 74 69 61 6c 69 74 65 22 2c 20 "initial state",

0006 22 70 61 72 61 6d 73 22 3a 20 7b 22 70 72 6f 74 "params": { "prot

0007 6f 63 6f 6c 56 65 72 73 69 6f 64 22 3a 20 22 32ocolVersion": 2

0008 00 12 05 20 31 2d 32 35 22 2c 20 22 63 61 70 "capabilities", "cap

0009 61 62 69 6c 69 74 69 65 73 22 3a 20 7b 7d 2c 20abilities": {}, "

000a 22 63 6c 69 65 64 74 69 66 6f 22 3a 20 7b 22 "clientInfo": {

000b 6e 61 6d 65 22 3a 20 22 61 63 31 30 36 37 2d 63name": "c3m07-c

000c 68 61 74 62 6f 74 2d 68 6f 73 74 22 2c 20 22 76hatbot-host": "v

000d 65 72 73 69 6f 64 22 3a 20 22 30 2e 31 2e 30 22ersion": "0.1.0"

000e 7d 7d 7d}

Mensaje initialize enviado por el chatbot

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
652	20:19:46.89688	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_	
653	20:19:46.89958	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
654	20:19:46.89786	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
655	20:19:46.89718	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD_	
656	20:19:46.89728	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
657	20:19:46.89722	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1 , JSON (application/json)	
658	20:19:46.89723	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=465 Win=65024 Len=0	
672	20:19:46.10383	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [PSH, ACK] Seq=1 Ack=465 Win=65024 Len=72 [TCP P_	
673	20:19:46.10385	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=73 Win=65280 Len=0	
674	20:19:46.10382	127.0.0.1	127.0.0.1	HTTP/1.1	200 OK , JSON (application/json)	
677	20:19:46.10390	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=246 Win=65280 Len=0	
678	20:19:46.10407	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [FIN, ACK] Seq=465 Ack=246 Win=65280 Len=0	
679	20:19:46.10409	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=246 Ack=466 Win=65024 Len=0	
680	20:19:46.10411	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [FIN, ACK] Seq=246 Ack=466 Win=65024 Len=0	
681	20:19:46.10413	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=466 Ack=247 Win=65280 Len=0	
684	20:19:46.10579	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_	
685	20:19:46.10587	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
686	20:19:46.10592	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
687	20:19:46.10602	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=280 [TCP PD_	
688	20:19:46.10604	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
689	20:19:46.10605	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1 , JSON (application/json)	
690	20:19:46.10606	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=352 Win=65024 Len=0	
699	20:19:46.11122	127.0.0.1	127.0.0.1	HTTP	HTTP/1.1 202 Accepted	
700	20:19:46.11124	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=352 Ack=45 Win=65280 Len=0	
703	20:19:46.11149	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [FIN, ACK] Seq=352 Ack=45 Win=65280 Len=0	

0000	02 00 00 00 45 00 00 d5 f1 6a 40 00 00 06 8a b6	...	E	...	...
0010	7f 00 00 01 7f 00 00 01 cc 8f 2f 53 a5 8c 8d 73	...	S	...	...
0020	c2 0f af 2f 50 18 00 ff 50 97 00 00 70 22 6a 7d	...	/P	...	...
0030	6f 64 72 70 63 22 3a 20 22 2a 30 22 2c 20 22	...	onrpc"	...	...
0040	6d 65 74 68 6f 64 21 3a 20 22 2e 6f 74 69 66 69	...	method":	...	...
0050	63 61 74 69 6f 6e 73 2f 69 6e 69 74 69 61 6c 69	...	ctions/	...	...
0060	74 65 64 22 2c 20 22 70 61 72 61 6d 73 22 3a 20	...	zed", "p	...	...
0070	70 7d 7d	...	}	...	...

Respuesta del servidor al mensaje initialize

Después de esto, el chatbot manda un segundo mensaje llamado notifications initialized, este mensaje solo avisa que la conexión ya quedó lista, no espera ninguna respuesta con contenido, el servidor solamente contesta con un código 202 y sin ningún cuerpo, estos dos mensajes son los que se consideran de sincronización, porque solo sirven para dejar lista la conexión, todavía no piden ninguna acción concreta.

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
652	20:19:46.89688	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_	
653	20:19:46.89958	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
654	20:19:46.89786	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
655	20:19:46.89718	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD_	
656	20:19:46.89728	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
657	20:19:46.89722	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1 , JSON (application/json)	
658	20:19:46.89723	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=465 Win=65024 Len=0	
672	20:19:46.10383	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=1 Ack=465 Win=65024 Len=72 [TCP P_	
673	20:19:46.10385	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=73 Win=65280 Len=0	
674	20:19:46.10382	127.0.0.1	127.0.0.1	HTTP/1.1	200 OK , JSON (application/json)	
677	20:19:46.10390	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=465 Ack=246 Win=65280 Len=0	
678	20:19:46.10407	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [FIN, ACK] Seq=465 Ack=246 Win=65280 Len=0	
679	20:19:46.10409	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [ACK] Seq=246 Ack=466 Win=65024 Len=0	
680	20:19:46.10411	127.0.0.1	127.0.0.1	TCP	8787 → 52365 [FIN, ACK] Seq=246 Ack=466 Win=65024 Len=0	
681	20:19:46.10413	127.0.0.1	127.0.0.1	TCP	52365 → 8787 [ACK] Seq=466 Ack=247 Win=65280 Len=0	
684	20:19:46.10579	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_	
685	20:19:46.10587	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
686	20:19:46.10592	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
687	20:19:46.10602	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=280 [TCP PD_	
688	20:19:46.10604	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
689	20:19:46.10605	127.0.0.1	127.0.0.1	HTTP/1.1	POST / HTTP/1.1 , JSON (application/json)	
690	20:19:46.10606	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=352 Win=65024 Len=0	
699	20:19:46.11122	127.0.0.1	127.0.0.1	HTTP	HTTP/1.1 202 Accepted	
700	20:19:46.11124	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=352 Ack=45 Win=65280 Len=0	
703	20:19:46.11149	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [FIN, ACK] Seq=352 Ack=45 Win=65280 Len=0	

0000	02 00 00 00 45 00 00 ff f1 77 40 00 00 06 80 0f	...	E	...	...
0010	7f 00 00 01 7f 00 00 01 cc 8f 2f 53 a5 8c 8d 73	...	S	...	...
0020	c2 0f af 2f 50 18 00 ff 50 97 00 00 70 22 6a 7d	...	/P	...	...
0030	6f 64 72 70 63 22 3a 20 22 2a 30 22 2c 20 22	...	onrpc"	...	...
0040	6d 65 74 68 6f 64 21 3a 20 22 2e 6f 74 69 66 69	...	method":	...	...
0050	63 61 74 69 6f 6e 73 2f 69 6e 69 74 69 61 6c 69	...	ctions/	...	...
0060	74 65 64 22 2c 20 22 70 61 72 61 6d 73 22 3a 20	...	zed", "p	...	...
0070	70 7d 7d	...	}	...	...

Mensaje notifications initialized enviado por el chatbot

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
688	20:19:46.10604.	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
689	20:19:46.10605.	127.0.0.1	127.0.0.1	HTTP/3.	POST / HTTP/1.1 , JSON (application/json)	
690	20:19:46.10606.	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=1 Ack=352 Win=65024 Len=0	
699	20:19:46.11120.	127.0.0.1	127.0.0.1	HTTP	HTTP/1.1 200 OK (application/json)	
700	20:19:46.11124.	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=352 Ack=45 Win=65280 Len=0	
703	20:19:46.11149.	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [FIN, ACK] Seq=352 Ack=45 Win=65280 Len=0	
704	20:19:46.11156.	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=45 Ack=353 Win=65024 Len=0	
705	20:19:46.11159.	127.0.0.1	127.0.0.1	TCP	8787 → 52367 [ACK] Seq=45 Ack=383 Win=65024 Len=0	
706	20:19:46.11162.	127.0.0.1	127.0.0.1	TCP	52367 → 8787 [ACK] Seq=353 Ack=46 Win=65280 Len=0	
709	20:19:46.11251.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1	
710	20:19:46.11256.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
711	20:19:46.11265.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
712	20:19:46.11276.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=280 [TCP PD.	
713	20:19:46.11277.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1 Ack=346 Win=65024 Len=0	
714	20:19:46.11279.	127.0.0.1	127.0.0.1	HTTP/3.	POST / HTTP/1.1 , JSON (application/json)	
715	20:19:46.11280.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1 Ack=346 Win=65024 Len=0	
726	20:19:46.11868.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [PSH, ACK] Seq=1 Ack=346 Win=65024 Len=73 [TCP P.	
727	20:19:46.11869.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=346 Ack=74 Win=65280 Len=0	
730	20:19:46.11875.	127.0.0.1	127.0.0.1	HTTP/3.	HTTP/1.1 200 OK , JSON (application/json)	
731	20:19:46.11876.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=346 Ack=1728 Win=63744 Len=0	
732	20:19:46.11892.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [FIN, ACK] Seq=346 Ack=1728 Win=63744 Len=0	
733	20:19:46.11894.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1728 Ack=347 Win=65024 Len=0	
734	20:19:46.11898.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [FIN, ACK] Seq=1728 Ack=347 Win=65024 Len=0	
735	20:19:46.11901.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=347 Ack=1729 Win=63744 Len=0	
1025	20:19:46.14981.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1	
1026	20:19:46.14989.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
1027	20:19:46.14997.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
1028	20:19:46.175007.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD.	
1029	20:19:46.175011.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=1 Ack=282 Win=65024 Len=0	
1030	20:19:46.175012.	127.0.0.1	127.0.0.1	HTTP/3.	POST / HTTP/1.1 , JSON (application/json)	
1031	20:19:46.175013.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=1 Ack=390 Win=65024 Len=0	
1045	20:19:46.175995.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [PSH, ACK] Seq=1 Ack=390 Win=65024 Len=72 [TCP P.	
1046	20:19:46.175997.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=73 Win=65280 Len=0	
1049	20:19:46.176003.	127.0.0.1	127.0.0.1	HTTP/3.	HTTP/1.1 200 OK , JSON (application/json)	
1050	20:19:46.176004.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=233 Win=65280 Len=0	
1051	20:19:46.176018.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [FIN, ACK] Seq=390 Ack=233 Win=65280 Len=0	
1052	20:19:46.176021.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=233 Ack=391 Win=65024 Len=0	

```

Transmission Control Protocol, Src Port: 8787, Dst Port: 52367, Seq: 1,
Source Port: 8787
Destination Port: 52367
[Stream index: 102]
[Stream Packet Number: 8]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 44]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 3255807791
[Next Sequence Number: 45 (relative sequence number)]
Acknowledgment Number: 352 (relative ack number)
Acknowledgment Number (raw): 2783874331
0101 .... = Header Length: 20 bytes (5)
Flags: 0x010 (PSH, ACK)
Window: 254
[Calculated window size: 65024]
[Window size scaling factor: 256]
Checksum: 0x1048 [Unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamp]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (44 bytes)
Hypertext Transfer Protocol

```

```

0000 02 00 00 00 45 00 00 54 f1 81 40 00 00 06 00 20 .....E.T. @...
0010 7f 00 00 01 7f 00 00 01 22 53 cc 8f c2 ef af 2f .....TS.../
0020 45 00 01 10 50 00 f4 10 40 00 00 40 54 54 50 ....P...H. HTTP
0030 2f 31 2e 31 2e 32 30 20 41 63 63 65 70 74 65 /./1.202 Accepte
0040 64 0d 0e 43 6f 64 74 65 6e 74 2d 4c 65 6e 67 74 d Conte nt-Lengt
0050 65 3a 20 30 0d 0a 0d 0a .....

```

Respuesta vacía del servidor, código 202, sin cuerpo

Mensajes de solicitud y de respuesta

Una vez que la conexión ya está lista, el chatbot manda el mensaje tools list, con este mensaje le pide al servidor la lista completa de herramientas que tiene disponibles, el servidor contesta con las seis herramientas de la clínica y los parámetros que necesita cada una.

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
712	20:19:46.11276.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=280 [TCP PD.	
713	20:19:46.11277.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
714	20:19:46.11279.	127.0.0.1	127.0.0.1	HTTP/3.	POST / HTTP/1.1 , JSON (application/json)	
715	20:19:46.11280.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1 Ack=346 Win=65024 Len=0	
726	20:19:46.11868.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [PSH, ACK] Seq=1 Ack=346 Win=65024 Len=73 [TCP P.	
727	20:19:46.11869.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=346 Ack=74 Win=65280 Len=0	
730	20:19:46.11875.	127.0.0.1	127.0.0.1	HTTP/3.	HTTP/1.1 200 OK , JSON (application/json)	
731	20:19:46.11876.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=346 Ack=1728 Win=63744 Len=0	
732	20:19:46.11892.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [FIN, ACK] Seq=346 Ack=1728 Win=63744 Len=0	
733	20:19:46.11894.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [ACK] Seq=1728 Ack=347 Win=65024 Len=0	
734	20:19:46.11898.	127.0.0.1	127.0.0.1	TCP	8787 → 52368 [FIN, ACK] Seq=1728 Ack=347 Win=65024 Len=0	
735	20:19:46.11901.	127.0.0.1	127.0.0.1	TCP	52368 → 8787 [ACK] Seq=347 Ack=1729 Win=63744 Len=0	
1025	20:19:46.14981.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=65495 WS=256 SACK_PERM=1	
1026	20:19:46.14989.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
1027	20:19:46.14997.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
1028	20:19:46.175007.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD.	
1029	20:19:46.175011.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=1 Ack=282 Win=65024 Len=0	
1030	20:19:46.175012.	127.0.0.1	127.0.0.1	HTTP/3.	POST / HTTP/1.1 , JSON (application/json)	
1031	20:19:46.175013.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=1 Ack=390 Win=65024 Len=0	
1045	20:19:46.175995.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [PSH, ACK] Seq=1 Ack=390 Win=65024 Len=72 [TCP P.	
1046	20:19:46.175997.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=73 Win=65280 Len=0	
1049	20:19:46.176003.	127.0.0.1	127.0.0.1	HTTP/3.	HTTP/1.1 200 OK , JSON (application/json)	
1050	20:19:46.176004.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=233 Win=65280 Len=0	
1051	20:19:46.176018.	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [FIN, ACK] Seq=390 Ack=233 Win=65280 Len=0	
1052	20:19:46.176021.	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=233 Ack=391 Win=65024 Len=0	

```

[Stream index: 102]
[Stream Packet Number: 6]
[Conversation completeness: Complete, WITH_DATA (31)]
[TCP Segment Len: 65]
Sequence Number: 281 (relative sequence number)
Sequence Number (raw): 2454588950
[Next Sequence Number: 346 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment Number (raw): 1187632100
0101 .... = Header Length: 20 bytes (5)
Flags: 0x010 (PSH, ACK)
Window: 255
[Calculated window size: 65280]
[Window size scaling factor: 256]
Checksum: 0x1e9f [Unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamp]
[SEQ/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (65 bytes)
TCP segment data (65 bytes)
[2 Reassembled TCP Segments (345 bytes): #712(280), #714(65)]
Hypertext Transfer Protocol
JavaScript Object Notation: application/json

```

```

0000 02 00 00 00 45 00 00 69 f1 90 40 00 00 06 0a fc .....E.i. @...
0010 7f 00 00 01 7f 00 00 01 cc 90 22 53 92 4c d5 96 .....TS...-S.Ln
0020 45 00 01 10 50 00 f4 10 40 00 00 70 72 74 75 ....P...-S.Ln
0030 6f 64 72 70 63 22 3a 20 22 32 2a 30 22 2c 20 22 snrpc": "2.0", "
0040 59 64 22 3a 20 32 2c 20 22 6d 65 74 68 6f 64 22 id": 2, "method"
0050 3a 20 22 74 6f 6f 66 73 2f 6f 69 73 74 22 2c 20 "tools list",
0060 22 70 61 72 61 64 73 22 3a 20 70 7d 7c .....

```

Mensaje tools list enviado por el chatbot

Respuesta del servidor con las seis herramientas

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
734	20:19:46.1189	127.0.0.1	127.0.0.1	TCP	727 -> 52368 [FIN, ACK] Seq=1728 Ack=347 Win=65024 Len=0	
735	20:19:46.1190	127.0.0.1	127.0.0.1	TCP	52368 -> 8787 [ACK] Seq=347 Ack=1728 Win=63744 Len=0	
1805	20:19:53.7408	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [SYN] Seq=0 Win=65535 Len=0 MSS=5459	
1806	20:19:53.7409	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [ACK] Seq=1 Ack=0 Win=65535 Len=0 MSS=5459	
1807	20:19:53.7499	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
1808	20:19:53.7500	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PO.]	
1809	20:19:53.7501	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
1810	20:19:53.7502	127.0.0.1	127.0.0.1	HTTP/1.1	POST /HTTP/1.1 300 OK - 350M (application/json)	
1811	20:19:53.7503	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [ACK] Seq=1 Ack=390 Win=65024 Len=0	
1845	20:19:53.7595	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [PSH, ACK] Seq=1 Ack=390 Win=65024 Len=7 [TCP P.]	
1846	20:19:53.7599	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [ACK] Seq=390 Ack=73 Win=65280 Len=0	
1849	20:19:53.7603	127.0.0.1	127.0.0.1	HTTP/1.1	300 OK - 350M (application/json)	
1850	20:19:53.7604	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [ACK] Seq=390 Ack=233 Win=65280 Len=0	
1851	20:19:53.7618	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [FIN, ACK] Seq=390 Ack=233 Win=65280 Len=0	
1852	20:19:53.7621	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [ACK] Seq=233 Ack=391 Win=65024 Len=0	
1853	20:19:53.7627	127.0.0.1	127.0.0.1	TCP	8787 -> 52378 [FIN, ACK] Seq=233 Ack=391 Win=58024 Len=0	
1854	20:19:53.7630	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [ACK] Seq=391 Ack=234 Win=65280 Len=0	
1855	20:19:53.7631	127.0.0.1	127.0.0.1	TCP	52378 -> 8787 [FIN, ACK] Seq=391 Ack=234 Win=65280 Len=0	
2290	20:19:56.2674	127.0.0.1	127.0.0.1	TCP	8787 -> 52424 [ACK] Seq=1 Ack=412 Win=65024 Len=0	
2291	20:19:56.2675	127.0.0.1	127.0.0.1	TCP	52424 -> 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
2292	20:19:56.2728	127.0.0.1	127.0.0.1	TCP	52424 -> 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PO.]	
2293	20:19:56.2728	127.0.0.1	127.0.0.1	TCP	8787 -> 52424 [ACK] Seq=1 Ack=281 Win=65024 Len=0	
2294	20:19:56.2727	127.0.0.1	127.0.0.1	HTTP/1.1	POST /HTTP/1.1 350M (application/json)	
2295	20:19:56.2728	127.0.0.1	127.0.0.1	TCP	8787 -> 52424 [ACK] Seq=1 Ack=412 Win=65024 Len=0	
2299	20:19:56.2795	127.0.0.1	127.0.0.1	TCP	8787 -> 52424 [PSH, ACK] Seq=1 Ack=412 Win=65024 Len=7 [TCP P.]	
9880	92 80 00 00 45 00 00 04	72 c4 00 00 80 86 09 97	E	S h-		
9810	7f 00 00 01 7f 00 00 01	cc 9a 22 53 02 68 2d d9	S h-			
9820	25 07 80 90 50 18 00 ff	4f c1 00 00 70 22 6a 73	S P L ("js			
9830	6f 63 00 00 00 00 00 00	22 22 22 22 22 22 22 22	comp: "2 0"			
9840	69 64 22 3a 20 33 2c 20	22 6d 65 7a 68 6f 64 22	id: "method"			
9850	3a 20 72 64 ff 6c 73 7f	63 63 61 6c 22 2c 20	tools: "call"			
9860	00 00 00 00 00 00 00 00	4a 20 70 22 6a 61 6d 65	param: " ("name"			
9870	22 3a 20 22 6c 69 73 7a	5f 73 70 65 61 69 61 6c	list: "special			
9880	74 69 65 67 22 2c 20 2d	61 72 67 75 6d 65 66 74	ties", argument			
9890	73 22 3a 20 70 7d 7d 70		a: (}}}			

Mensaje tools call enviado por el chatbot, con las capas de la comunicación expandidas

No.	Time	Source	Destination	Protocol	Info	longitud de protocolo
1030	20:19:53.75012	127.0.0.1	127.0.0.1	HTTP/3..	POST / HTTP/1.1 , 350M (application/json)	
1031	20:19:53.75013	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=1 Ack=390 Win=65024 Len=0	
1045	20:19:53.75995	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [PSH, ACK] Seq=1 Ack=390 Win=65024 Len=72 [TCP P..	
1046	20:19:53.75997	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=73 Win=65280 Len=0	
1049	20:19:53.76000	127.0.0.1	127.0.0.1	HTTP/3..	HTTP/1.1 200 OK , 350M (application/json)	
1050	20:19:53.76004	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=390 Ack=233 Win=65280 Len=0	
1051	20:19:53.76018	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [FIN, ACK] Seq=390 Ack=233 Win=65280 Len=0	
1052	20:19:53.76021	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [ACK] Seq=233 Ack=391 Win=65024 Len=0	
1053	20:19:53.76027	127.0.0.1	127.0.0.1	TCP	8787 → 52378 [FIN, ACK] Seq=233 Ack=391 Win=65024 Len=0	
1054	20:19:53.76030	127.0.0.1	127.0.0.1	TCP	52378 → 8787 [ACK] Seq=391 Ack=234 Win=65280 Len=0	
2249	20:19:53.76073	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [PSH, ACK] Seq=412 Ack=320 Win=65024 Len=0	
2250	20:19:53.76749	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=65495	
2251	20:19:53.76757	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [ACK] Seq=1 Ack=1 Win=65280 Len=0	
2252	20:19:53.77082	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=281 [TCP PD..	
2253	20:19:53.77205	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [ACK] Seq=1 Ack=282 Win=65024 Len=0	
2254	20:19:53.77207	127.0.0.1	127.0.0.1	HTTP/3..	POST / HTTP/1.1 , 350M (application/json)	
2255	20:19:53.77208	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [ACK] Seq=1 Ack=412 Win=65024 Len=0	
2260	20:19:53.77953	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [PSH, ACK] Seq=1 Ack=412 Win=65024 Len=72 [TCP P..	
2310	20:19:53.77955	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [ACK] Seq=412 Ack=73 Win=65280 Len=0	
2313	20:19:53.77960	127.0.0.1	127.0.0.1	HTTP/3..	HTTP/1.1 200 OK , 350M (application/json)	
2314	20:19:53.77961	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [ACK] Seq=412 Ack=320 Win=65024 Len=0	
2315	20:19:53.77970	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [ACK] Seq=412 Ack=320 Win=65024 Len=0	
2316	20:19:53.77979	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [ACK] Seq=320 Ack=413 Win=65024 Len=0	
2317	20:19:53.77982	127.0.0.1	127.0.0.1	TCP	8787 → 52424 [FIN, ACK] Seq=320 Ack=413 Win=65024 Len=0	
2318	20:19:53.77985	127.0.0.1	127.0.0.1	TCP	52424 → 8787 [ACK] Seq=413 Ack=321 Win=65024 Len=0	

Transmission Control Protocol, Src Port: 8787, Dst Port: 52378, Seq: 73, Win: 65535, Len: 0
Source Port: 8787
Destination Port: 52378
[Stream index: 123]
[Stream Packet Number: 10]
[Conversation completeness: Complete, WITH_DATA (33)]
[TCP Segment Len: 160]
Sequence Number: 72 (relative sequence number)
Sequence Number (raw): 633833443
[Next Sequence Number: 233 (relative sequence number)]
Acknowledgment Number: 390 (relative ack number)
Acknowledgment Number (raw): 208154172
0101 = Header Length: 20 bytes (5)
Flags: 0x018 (PSH, ACK)
Window: 254
[Calculated window size: 65024]
[Window size scaling factor: 256]
Checksum: 0x0096 [unverified]
[Checksum Status: Unverified]
Urgent Pointer: 0
[Timestamps]
[SYN/ACK analysis]
[Client Contiguous Streams: 1]
[Server Contiguous Streams: 1]
TCP payload (160 bytes)
TCP segment data (160 bytes)
[2 Reassembled TCP Segments (232 bytes): #1045(72), #1049(160)]

```

0000  02 00 00 00 45 00 00 c8 f2 dd 40 00 80 06 09 50      E . . . . P
0010  7f 00 00 01 7f 00 00 01 22 53 25 c7 87 43          . . . . .
0020  61 00 24 3c 52 10 00 f4 0a 90 00 80 70 22 6a 73    h.p . . . . .
0030  6f 60 72 70 63 22 3a 22 32 2e 30 22 2c 22 69 64    onspc:" 2.0" ;d
0040  22 3a 33 2c 22 72 65 73 75 6c 74 22 3a 70 22 63  "3"res ult":{"c
0050  6f 60 74 65 64 74 22 3a 50 70 22 74 70 65 22     ontent":{"type
0060  3a 22 74 65 78 74 22 2c 22 74 65 70 74 22 3a 22  "text": "text";
0070  70 5c 22 73 70 65 63 69 61 6c 74 69 65 73 5c 22  ("speci alties"
0080  3a 30 2c 22 61 61 61 65 60 6f 66 6f 69 61 5c    [{"card iologia
0090  22 2c 5c 22 64 65 72 60 61 74 6f 6f 6f 69 61    }, "derm atologia
00a0  5c 22 2c 5c 22 70 65 64 69 61 74 72 69 61 5c 22  ", "ped iatria"
00b0  2c 5c 22 68 65 64 69 61 69 6a 61 5f 6f 65 6e 65  ", "medic ina gene
00c0  72 61 6c 5c 22 54 70 22 70 5d 70 70            ral""]" ]]]]

```

Respuesta del servidor con el resultado de la herramienta

Análisis por capas

En la captura se puede ver la comunicación pasando por varias capas, en la capa de enlace no aparece una tarjeta de red real, porque el tráfico va por el adaptador de loopback de la computadora, este adaptador simula una tarjeta de red pero solamente para tráfico que se queda dentro de la misma máquina.

En la capa de red se ve que tanto el origen como el destino son la misma dirección, 127.0.0.1, ya que el chatbot y el servidor están corriendo en la misma computadora.

En la capa de transporte se usa TCP, se puede ver el saludo de tres pasos al inicio de cada mensaje, con las banderas SYN, luego SYN ACK, luego ACK, y al final de cada intercambio se ve el cierre de la conexión con las banderas FIN ACK, algo importante es que cada mensaje JSON-RPC abre su propia conexión TCP nueva, se manda el mensaje, se recibe la respuesta, y esa conexión se cierra, la siguiente vez que se manda otro mensaje se abre una conexión distinta.

En la capa de aplicación se usa HTTP, cada mensaje JSON-RPC viaja dentro del cuerpo de una petición POST, con el encabezado Content-Type puesto como application json, y la respuesta llega como un HTTP 200 con el resultado en formato JSON, cuando el mensaje es una notificación, como notifications initialized, la respuesta es un HTTP 202 sin ningún contenido dentro.

Dificultades y lecciones aprendidas

Durante el proyecto salieron varios problemas prácticos que no se ven en la documentación del protocolo.

El servidor oficial de git no tiene ninguna herramienta para crear un repositorio nuevo, para resolver esto el chatbot crea ese repositorio una sola vez cuando arranca, y a partir de ahí todo lo demás, como crear el archivo y hacer el commit, se hace por medio de la conversación normal con el modelo.

En la terminal de PowerShell las letras con tilde y la letra ñ se veían mal, esto se arregló diciéndole a Python que use UTF-8 para escribir en la consola desde el inicio del programa.

También salieron problemas de red, la conexión de internet que se usó intercepta el tráfico HTTPS, esto hacía que a veces las conexiones fallaran por errores de certificado, para resolver esto se usó una librería que valida los certificados contra el almacén de certificados de Windows, y también se agregaron varios reintentos con espera entre cada uno.

Cloudflare contestaba con un error 403 cuando la petición no traía un User Agent parecido al de un navegador normal, esto se resolvió mandando siempre un User Agent como el de Chrome.

El primer intento de desplegar el servidor remoto no funcionó porque la cuenta de Cloudflare todavía no tenía un subdominio registrado, esto se resolvió registrando un subdominio desde la página de Cloudflare, este paso solo se puede hacer desde la página web, no hay ningún comando para hacerlo.

Por último, al querer descifrar el tráfico HTTPS para poder verlo en Wireshark, la función que trae Python para eso hacía que el programa se cerrara solo, sin ningún mensaje de error, esto pasa por un problema conocido de Python en Windows, por esta razón se decidió correr el servidor remoto de forma local y sin HTTPS, solamente para poder hacer esta captura.

Con todo esto se aprendió que MCP es un protocolo bastante simple en su forma de mandar mensajes, pero que conectar todas las piezas, el modelo, el chatbot y los distintos servidores, trae varios problemas prácticos que no se ven a simple vista en la documentación, también se aprendió a leer tráfico de red con Wireshark, y a entender cómo un protocolo de aplicación como JSON-RPC viaja arriba de TCP y de HTTP.

Conclusiones

Con este proyecto se logró construir un chatbot completo que puede hablar con un modelo de lenguaje y que además puede usar herramientas externas por medio de MCP, se cumplió con escribir todo el protocolo JSON-RPC a mano, sin usar ningún SDK de MCP, y se logró conectar tanto servidores oficiales de Anthropic como un servidor propio, este servidor propio se probó funcionando de forma local y también de forma remota en la nube.

También se logró usar Wireshark para ver cómo viajan los mensajes de este protocolo por la red, y separar cuáles mensajes son de sincronización, cuáles son de solicitud y cuáles son de respuesta.

Este proyecto ayudó a entender que MCP no es algo complicado, es solamente una forma ordenada de mandar mensajes en formato JSON para que un modelo de lenguaje pueda pedirle acciones a un programa externo, y que buena parte del trabajo real de un proyecto como este está en manejar los detalles de la conexión, la red, y los errores que van saliendo en el camino, más que en el protocolo en sí mismo.